

컴퓨터비전_인공지능(AI)개론

11. CNN 기반 이미지분류

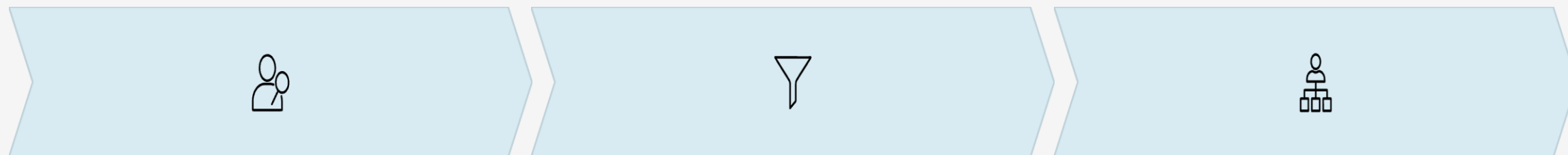
- 문제 정의하기
- CNN의 처리 개요
- 파이토치에서 CNN을 구현하는 방법
- 공통 함수 사용하기
- 데이터 준비
- 모델 정의(전결합형)
- 결과(전결합형)
- 모델 정의(CNN)
- 결과(CNN)

우리가 눈으로 사물을 볼 때, 단순히 픽셀의 나열을 보는 게 아니라 선, 모서리, 질감 같은 특징들을 조합해서 '이건 고양이구나', '저건 자동차구나' 하고 인식합니다.

합성곱 신경망(Convolutional Neural Network, CNN)은 바로 이 방식을 흉내 낸 딥러닝 모델입니다.

이전까지 다른 신경망은 이미지를 길게 펼친 1차원 데이터로 다루다 보니, 픽셀들이 서로 어떻게 이웃하고 있는지, 즉 이미지의 중요한 공간적 정보를 잃어버리는 한계가 있었습니다.

CNN은 이미지의 공간 구조를 그대로 유지한 채, 마치 돋보기로 특징을 살펴보듯이 작동하여 이미지 인식을 진행합니다.



특징 추출

이미지에서 중요한 패턴과 특징을
찾아내는 단계

정보 압축

핵심 정보만 남기고 불필요한 세부사항
제거

최종 분류

추출된 특징을 바탕으로 이미지가
무엇인지 판단

CNN은 크게 두 부분으로 나눌 수 있습니다.

- 이미지의 특징을 추출하는 **특징 추출(Features)** 부분
- 추출된 특징을 보고 이미지가 무엇인지 분류하는 **분류(Classifier)** 부분입니다.

CNN을 레고 조립 전문가에 비유하면 이해하기 쉽습니다.

01

합성곱 (Convolution)

전문가는 먼저 수많은 레고 블록 속에서 '1x2 크기의 빨간 블록', '동그란 투명 블록'처럼 특정 모양과 색의 기본 부품(특징)들을 찾아냅니다. 이때 사용하는 '특징을 찾아내는 툴'이 바로 커널(Kernel)입니다.

02

풀링 (Pooling)

각 구역에서 가장 대표적인 부품 하나씩만 뽑아 전체적인 부품의 수를 줄이고, 어떤 부품들이 있는지만 간략하게 요약합니다.

03

전결합층 (Fully Connected Layer)

모인 핵심 부품들의 조합을 보고, "아, 이 부품들로는 자동차를 만들 수 있겠군!"하고 최종 결론을 내리는 단계입니다.

nn.Conv2d의 작동 원리

작은 크기의 필터인 커널(Kernel)이 이미지 위를 한 칸씩 이동하면서, 이미지의 특정 부분과 커널의 값들을 곱하고 모두 더해 하나의 결과값을 만듭니다.

이 과정을 통해 이미지의 모서리, 질감, 색상 패턴 등과 같은 저수준(low-level) 특징을 추출할 수 있습니다.

중요한 점: 이 커널의 값들이 바로 학습을 통해 최적화되는 파라미터(가중치)라는 것입니다.

핵심 포인트

- 커널이 이미지를 스캔
- 특징 맵(Feature Map) 생성
- 학습 가능한 파라미터
- 공간 정보 유지



크기 축소

특징 맵의 크기를 줄여 계산량 감소



강인함 확보

위치 변화에 대응할 수 있는 robustness 획득



최대 풀링

특정 구역(예: 2x2)에서 가장 큰 값만 선택

nn.MaxPool2d는 합성곱을 거친 특징 맵의 크기를 줄여주는 과정입니다.

예를 들어 최대 풀링(Max Pooling)은 특정 구역에서 가장 큰 값(가장 두드러진 특징)만 남기고 나머지는 버립니다. 이를 통해 이미지 내에서 특징의 위치가 약간 바뀌더라도 대응할 수 있는 강인함을 얻게 됩니다.

다차원에서 1차원으로

합성곱과 풀링을 거친 데이터는 여전히 채널, 가로, 세로 정보를 가진 다차원 텐서입니다.

하지만 최종 분류를 담당하는 선형 함수(`nn.Linear`)는 1차원 벡터 형태의 입력을 받습니다.

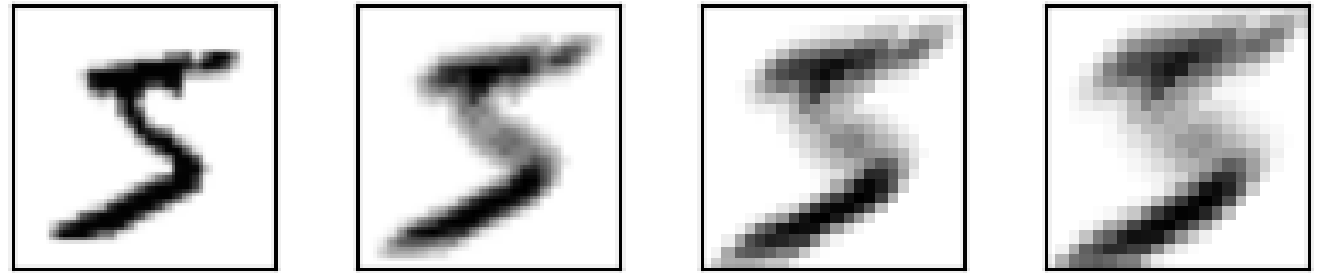
`nn.Flatten`은 이 다차원 텐서를 1차원으로 쭉 펼쳐주는 다리 역할을 합니다.

MNIST의 손글씨 숫자 데이터를 가지고 합성곱 처리를 실습해보겠습니다.

대각선상에만 가중치를 갖는 특수한 합성곱 함수를 만들어 손글씨 숫자에 3번 합성곱 처리를 적용합니다.

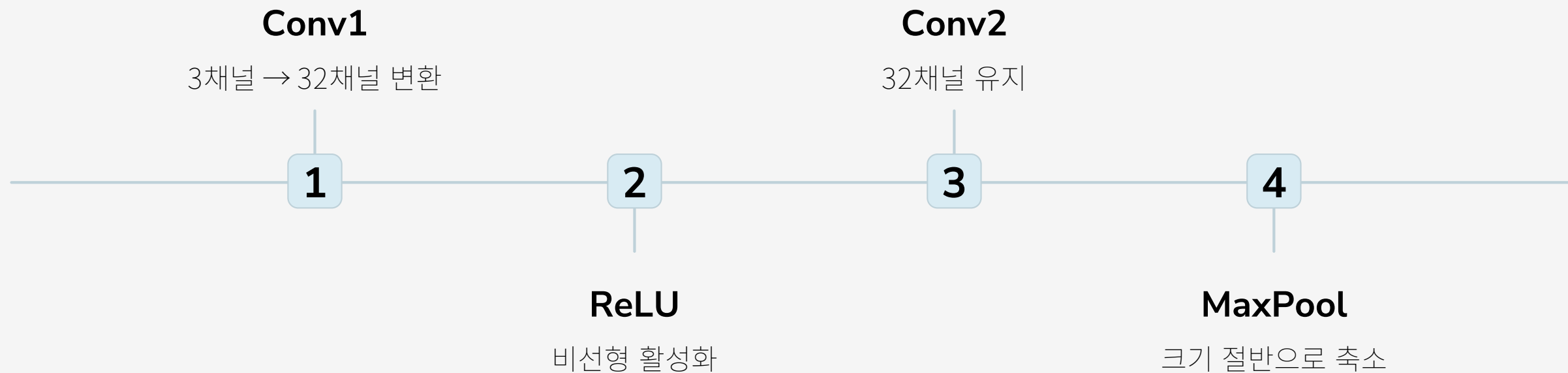
```
conv1 = nn.Conv2d(1, 1, 3) # 입력채널 : 1, 출력채널 : 1, 커널 크기 : 3x3
```

```
image, label = train_set[0]  
image = image.view(1,1,28,28)  
w1 = conv1(image)  
w2 = conv1(w1)  
w3 = conv1(w2)  
images = [image, w1, w2, w3]
```



결과를 시각화하면 합성곱을 거칠수록 이미지의 특정 특징이 강조되는 것을 확인할 수 있습니다.

```
# CNN 모델 전반 부분, 레이어 함수 정의
conv1 = nn.Conv2d(3, 32, 3) # 입력 채널: 3, 출력 채널: 32, 커널 크기: 3x3
relu = nn.ReLU(inplace=True) # 활성화 함수 ReLU
conv2 = nn.Conv2d(32, 32, 3) # 입력 채널: 32, 출력 채널: 32, 커널 크기: 3x3
maxpool = nn.MaxPool2d((2, 2)) # 풀링 영역 크기: 2x2
```



conv1과 conv2는 학습 가능한 파라미터(weight와 bias)를 가지고 있습니다.

레이어	Weight Shape	Bias Shape
conv1	[32, 3, 3, 3]	[32]
conv2	[32, 32, 3, 3]	[32]

conv1.weight의 shape [32, 3, 3, 3]은 [출력 채널 수, 입력 채널 수, 커널 높이, 커널 너비]를 의미합니다.

즉, 32개의 특징을 각각 추출하기 위해, 3개의 입력 채널(RGB) 각각에 적용될 3x3 크기의 커널 32세트가 준비되어 있다는 뜻입니다.

무작위 값으로 채워진 텐서를 만들어 각 레이어를 통과할 때마다 데이터의 shape이 어떻게 변하는지 추적해봅니다.

Input: [100, 3, 32, 32]

100개 이미지, 3채널(RGB), 32x32 크기

Conv1: [100, 32, 30, 30]

채널 3→32, 크기 32→30 (커널 적용)

Conv2: [100, 32, 28, 28]

채널 유지, 크기 30→28 (커널 적용)

MaxPool: [100, 32, 14, 14]

채널 유지, 크기 절반으로 축소

- ❏ 핵심 관찰: ReLU 활성화 함수는 값만 바꾸므로 shape은 변하지 않습니다.
2x2 풀링을 거치면서 이미지의 가로, 세로 크기가 정확히 절반으로 줄어듭니다.

기존 방식

```
x1 = conv1(inputs)
x2 = relu(x1)
x3 = conv2(x2)
x4 = relu(x3)
x5 = maxpool(x4)
```

여러 줄에 걸쳐 변수를 할당하며 계산

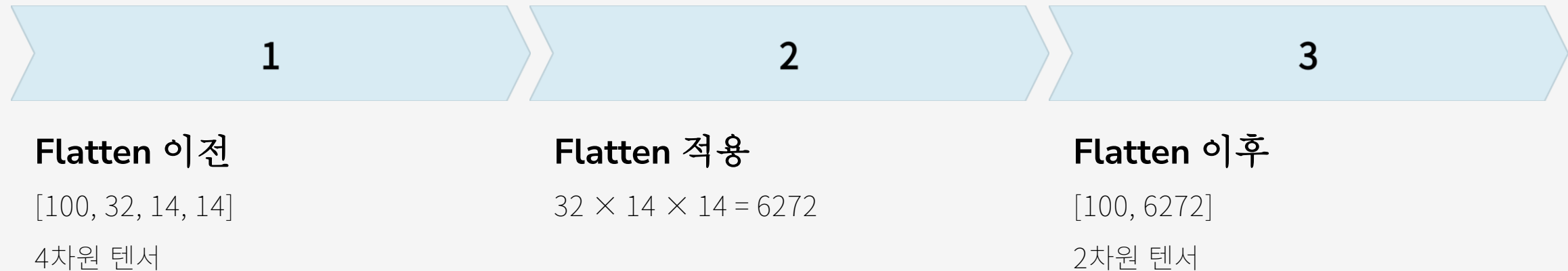
Sequential 사용

```
features = nn.Sequential(
    conv1,relu,conv2,relu,maxpool
)
```

훨씬 깔끔하고 읽기 쉬운 코드

레이어를 순차적으로 적용할 때는 **nn.Sequential** 컨테이너를 사용하면 코드를 훨씬 깔끔하게 만들 수 있습니다.

특징 추출부(features)를 통과한 다차원 텐서를 분류기(classifier)에 넣기 위해 1차원으로 펼칩니다.



데이터 건수(100)를 제외한 나머지 차원이 모두 곱해져 하나의 차원으로 합쳐졌습니다.

이제 이 데이터는 `nn.Linear(in_features=6272, ...)`와 같은 선형 함수에 입력될 준비가 되었습니다.

더 현실적이고 복잡한 문제인 컬러 이미지 분류를 진행하기 위해, 우선 이미 알고 있는 완전결합 신경망(Fully-Connected Network)을 먼저 사용해 보겠습니다.

데이터 전처리

`transforms.Compose`를 사용해 이미지를
모델에 맞게 가공

1계 텐서화

`view(-1)`로 3차원 이미지를 1차원 벡터로 변환

모델링 및 평가

`nn.Module` 상속, 손실 함수와 옵티마이저 설정

완전결합 신경망은 간단하지만, 픽셀 간의 공간적 관계 정보를 잃어버리는 단점이 있습니다.

완전결합 모델과 CNN 모델을 위해 두 가지 데이터 변환 파이프라인을 준비합니다.

Transform1: 전결합형용

```
transform1 = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(0.5, 0.5),
    transforms.Lambda(lambda x: x.view(-1)),
])
```

이미지를 1차원 텐서로 변환

결과: [3072] shape

Transform2: CNN용

```
transform2 = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(0.5, 0.5),
])
```

3차원 텐서 구조 유지

결과: [3, 32, 32] shape

CIFAR-10은 10개 클래스의 32x32 컬러 이미지로 구성된 데이터셋입니다.



Plan

비행기



Bird

새



Deer

사슴



Frog

개구리



Ship

배



Car

자동차



Cat

고양이



Dog

개



Horse

말



Truck

트럭

전결합형 모델 학습

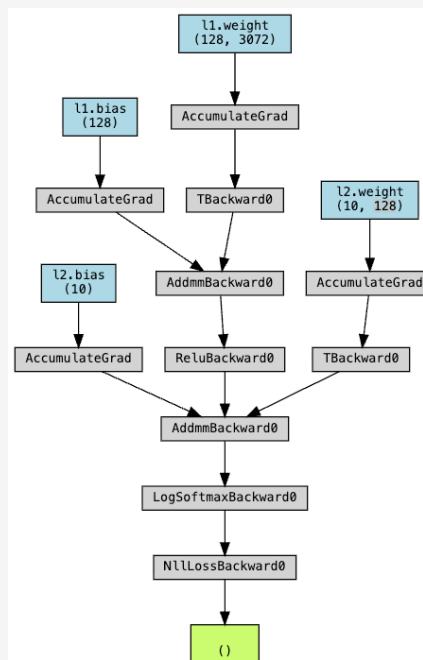
```
# 모델 인스턴스 생성
net = Net(n_input, n_output, n_hidden).to(device)

# 손실 함수 : 교차 엔트로피 함수
criterion = nn.CrossEntropyLoss()

# 학습률
lr = 0.01

# 최적화 함수: 경사 하강법
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

손실 계산 그래프



37.7% 53.0%

초기 정확도

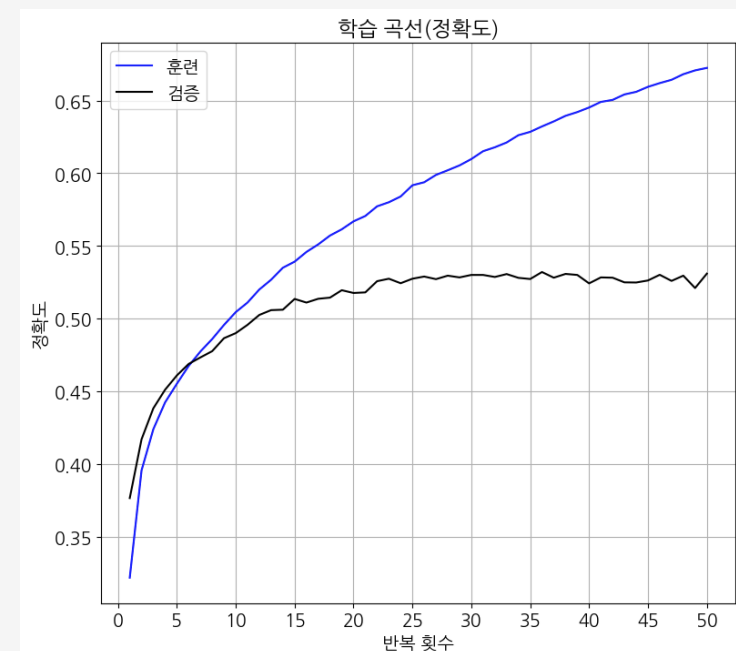
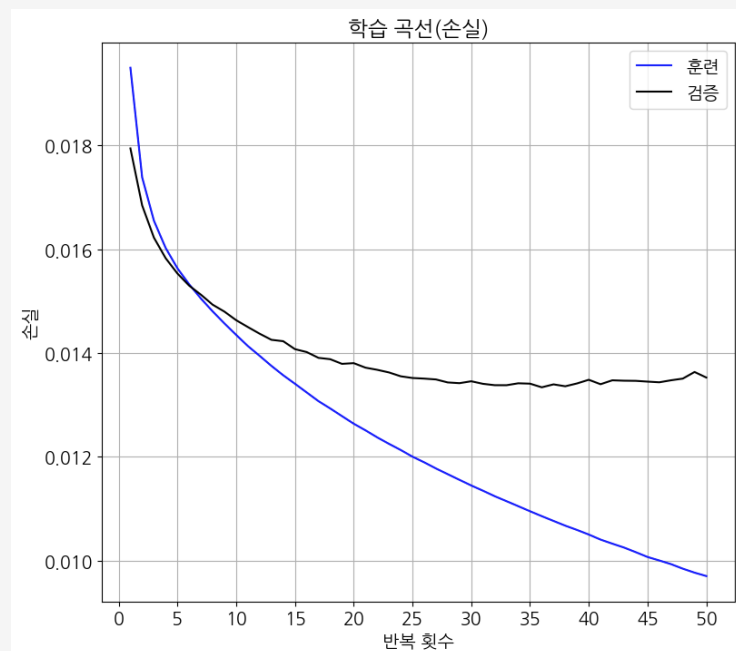
학습 시작 시점의

검증 정확도

최종 정확도

50 에폭 학습 후

검증 정확도



학습 곡선 그래프를 보면, 검증 데이터에 대한 정확도는 최대 53% 정도에 그치며 약 30 에폭 부근에서 학습이 정체되는 것을 알 수 있습니다. 10개 클래스를 무작위로 찍을 때의 정확도인 10%보다는 훨씬 높지만, 실용적으로 사용하기에는 아쉬운 성능입니다.



한계의 원인: 이미지를 1차원으로 펼치면서 이미지의 중요한 공간 구조 정보를 잃어버렸기 때문입니다.

이미지의 2D 구조를 그대로 존중하며 특징을 추출하는 CNN 모델을 파이토치로 직접 설계합니다.

```
class CNN(nn.Module):
    def __init__(self, n_output, n_hidden):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3) # 입력 채널:3, 출력 채널:32, 커널:3x3
        self.conv2 = nn.Conv2d(32, 32, 3) # 입력 채널:32, 출력 채널:32, 커널:3x3
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d((2,2))
        self.flatten = nn.Flatten()
        self.l1 = nn.Linear(6272, n_hidden) # 입력:6272, 출력:은닉노드 수
        self.l2 = nn.Linear(n_hidden, n_output) # 입력:은닉노드 수, 출력:클래스 수

        # 역할별로 그룹화
        self.features = nn.Sequential(
            self.conv1, self.relu, self.conv2, self.relu, self.maxpool)

        self.classifier = nn.Sequential(self.l1, self.relu, self.l2)

    def forward(self, x):
        x1 = self.features(x) # 특징 추출
        x2 = self.flatten(x1) # 1차원으로 펼치기
        x3 = self.classifier(x2) # 최종 분류
        return x3
```

__init__

모델에 필요한 모든 부품(레이어)들을 정의하고 초기화하는 단계 nn.Conv2d, nn.Linear 등 사용할 모든 레이어를 self.이름 형태로 선언합니다.

forward:

만든 부품들을 조립하여 데이터가 흘러갈 길을 정의하는 단계입니다.

입력 데이터 x가 어떤 레이어를 어떤 순서로 통과하여 최종 출력값을 만들지 결정합니다.

nn.Sequential

관련된 레이어들을 컨테이너로 묶어 features(특징 추출)와 classifier(분류)로 역할을 나눕니다.

CNN 설계 시 어려운 부분은 features 모듈을 통과한 데이터가 classifier의 첫 번째 Linear 레이어에 들어갈 때, 그 크기가 얼마인지 계산하는 것입니다.



CNN 모델 학습

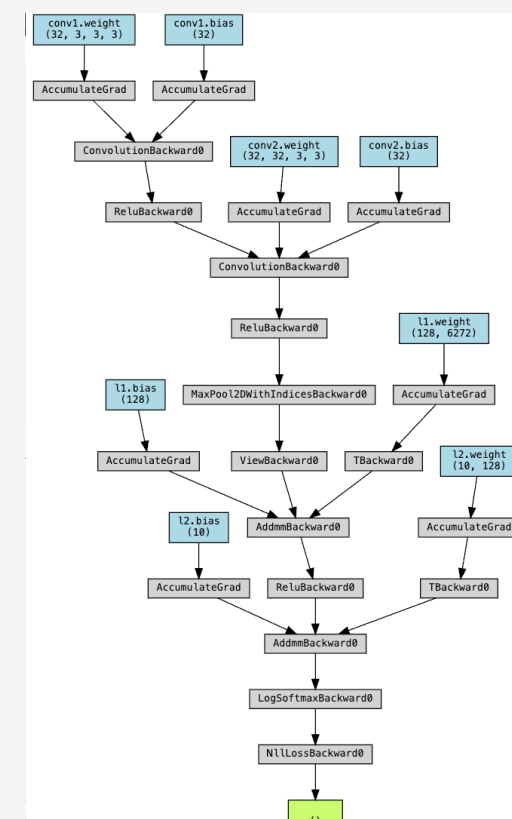
```
# 모델 인스턴스 생성
net = CNN(n_output, n_hidden).to(device)

# 손실 함수 : 교차 엔트로피 함수
criterion = nn.CrossEntropyLoss()

# 학습률
lr = 0.01

# 최적화 함수: 경사 하강법
optimizer = torch.optim.SGD(net.parameters(),
                               lr=lr)
```

손실 계산 그래프



34.6% **66.3%**

초기 정확도

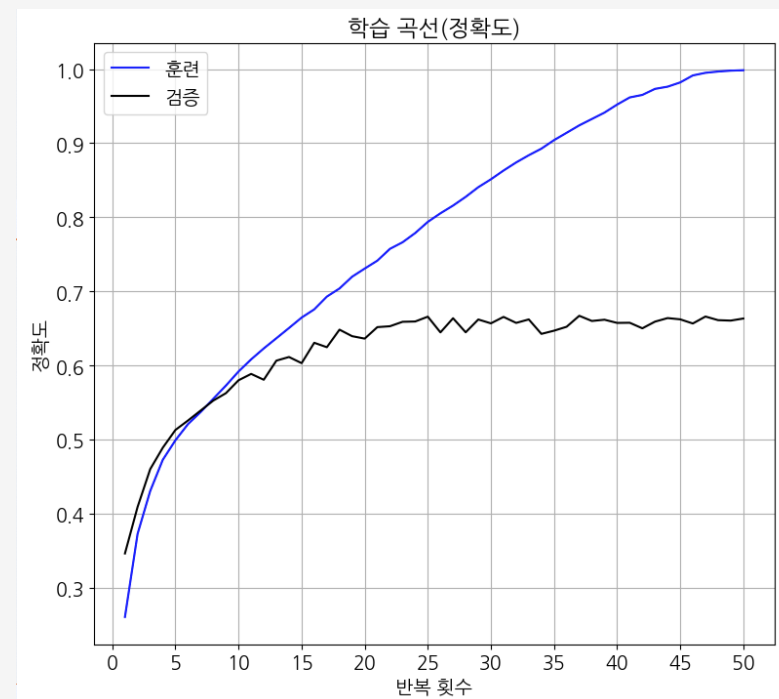
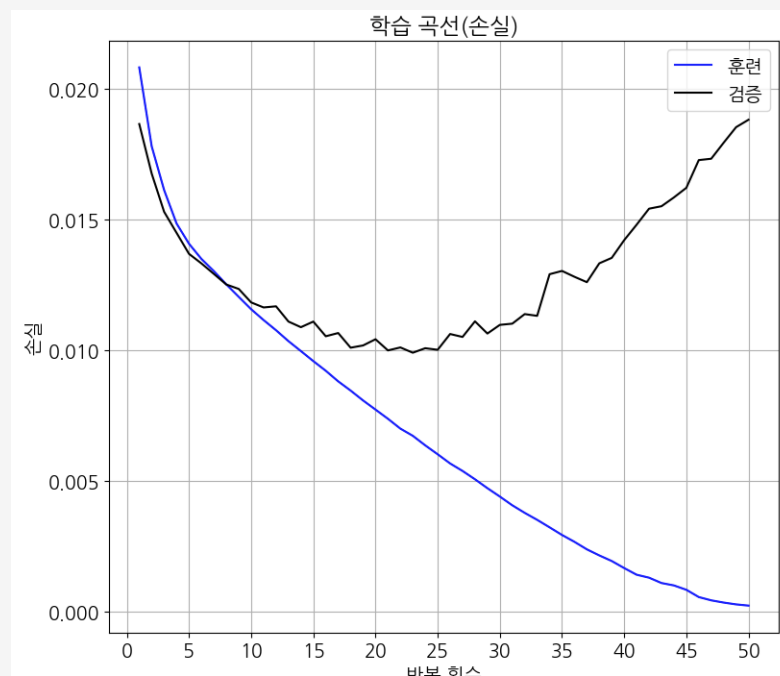
학습 시작 시점의

검증 정확도

최종 정확도

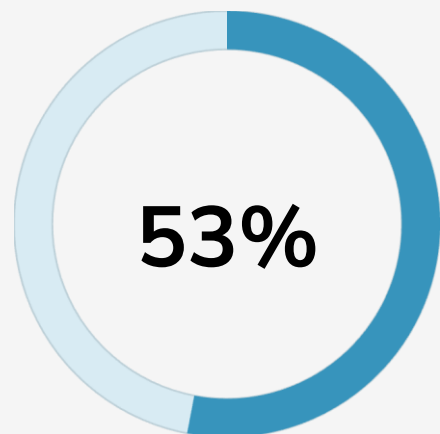
50 에폭 학습 후

검증 정확도



이미지의 공간 정보를 활용한 CNN이 전결합형 모델보다 13%p 더 높은 성능을 보였습니다.
다만 과적합 현상이 관찰되므로, 실제로는 20 에폭 근처에서 학습을 멈추는 것이 더 좋습니다.

완전결합 신경망

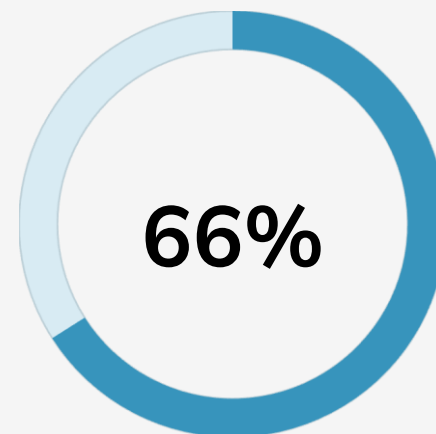


최종 정확도

공간 정보 손실로 인한 한계

- 이미지를 1차원으로 펼침
- 공간적 관계 정보 손실
- 30 에폭에서 학습 정체
- 실용성 부족

CNN 모델



최종 정확도

13%p 성능 향상

- 공간 구조 유지
- 특징 추출 효과적
- 20 에폭 이후 과적합 관찰
- 조기 종료 필요